zPSC Calibration and Test Manual

Michael Capotosto

May 2026

Contents

1 Introduction **1**

2 Cable Assemblies and Harnesses **3**
2.1 Overview . . . 3
2.1.1 2-Channel DCCT Cable . . . 3
2.1.2 4-Channel DCCT Cable . . . 4
2.1.3 Power Supply Channels 1 through 4 ATE Interface Cables . . . 4
2.1.4 SFP Transceivers . . . 4
2.1.5 Other Miscellaneous Cables . . . 5

3 ATE Connections **7**
3.1 Overview . . . 7
3.1.1 ATE Test Rack at NSLS-II . . . 9
3.2 Connections . . . 10
3.2.1 Current Measurement Loop/DMM Connection . . . 12

4 zPSC Chassis Overview **13**
4.1 Introduction . . . 13
4.2 Front Panels (2- and 4-Channel PSCs) . . . 13
4.3 Rear Panel - 2 Channel . . . 13
4.4 Internal Component Layout - 2 Channel . . . 14
4.4.1 Analog Board Components . . . 15
4.5 Rear Panel - 4 Channel . . . 16
4.6 Internal Component Layout - 4 Channel . . . 16
4.6.1 Analog Board Components . . . 17

5 EPICS Base and IOCs **19**
5.1 Introduction . . . 19
5.2 EPICS Installation Overview . . . 19
5.3 Installing EPICS Base and Required Modules . . . 19
5.4 Installing the ATE IOC . . . 21
5.5 Installing the PSC IOC . . . 22

6 CS-Studio - Phoebus **25**
6.1 Introduction . . . 25
6.2 Installing Phoebus . . . 25

7 Networking **27**
7.1 Overview . . . 27

8 Installing and Configuring Minicom **29**
8.1 Overview . . . 29
8.2 Installation in a Linux environment . . . 29
8.2.1 Installing Minicom . . . 29
8.3 Configuring Minicom . . . 30
8.4 Running Minicom . . . 30

9 Python Installation and Configuration **31**
9.1 Overview . . . 31
9.2 Installing and Configuring Python, and the Python Environment . . . 31
9.2.1 Installing Python . . . 31
9.2.2 Python Package Management . . . 31

10 Calibration and Test Script Installation **33**
10.1 Overview . . . 33
10.2 Cloning the Script Repository . . . 33
10.3 Creating the Virtual Environment . . . 33
10.4 Installing Dependencies . . . 33

11 Initial Booting of the PSC and Configuration of the EEPROM **35**
11.1 Overview . . . 35

12 Running the Calibration and Test Script **37**
12.1 Overview . . . 37
12.2 Running the Script: Launcher Menu . . . 37
12.2.1 Option 1: Initialize QSPI Only . . . 39
12.2.2 Option 2: Calibrate Only . . . 40

Chapter 1

Introduction

Chapter 2

Cable Assemblies and Harnesses

2.1 Overview

This chapter provides a brief overview of the makeup of various cable assemblies used in the sections to follow. Some cables are off-the-shelf cables with gender changers installed on one end. Others are made up of commercial cable assemblies with fly leads that must connect to breakout boards.

Some off the shelf fiber and network patch cables will be required.

2.1.1 2-Channel DCCT Cable

This cable consists of a Molex Micro-D15 cable, 72" long, fly lead, connecting to a Winford HD26 breakout board.

- 1x Molex 834229018 72" Micro-D15 Fly Lead Cable
- 1x Winford BRKEGD26HDF-T HD26F Breakout Board

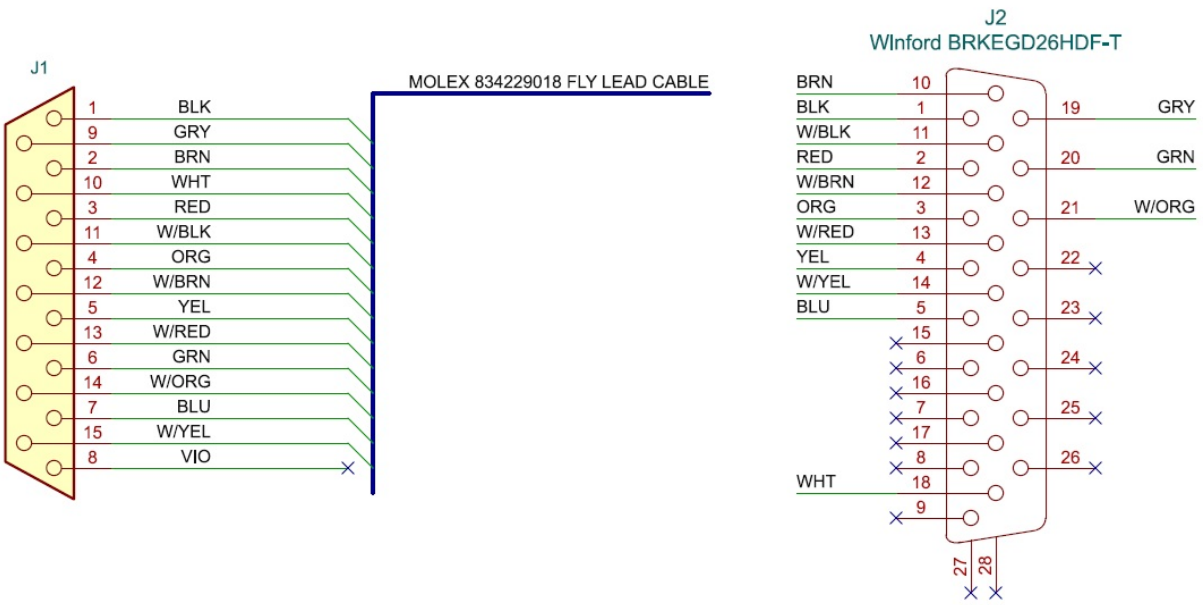


Figure 2.1: Pinout of 2-Channel microD15 to HD26 Cable

2.1.2 4-Channel DCCT Cable

This cable assembly consists of two parts:

- 1x Amphenol CS-DSDHD26MF0-005 1.52m HD26 M/F Cable
- 1x NorComp Inc. GCHDLP26F26F HD26 F/F Gender Changer

2.1.3 Power Supply Channels 1 through 4 ATE Interface Cables

This cable assembly consists of two parts:

- 4x Molex 83424-9062 Micro-D25 to DB25
- 4x L-com DGB25F DB25F/F Gender Changer

2.1.4 SFP Transceivers

Quantities depend on unit type and test workstation configuration. One SFP port is used for timing/event code tests on all units.

If testing FOFB, two more SFPs are required - the NSLS-II test stand uses one active copper and one fiber; though the simplest test setup would use two active copper SFPs.

- Fiber SFP: ATGBICS AFBR-57D7APZ-C or equivalent (8.5G+ SFP+, 850nm MM)
- Active Copper SFP: Finisar FCLF8522P2BTL or equivalent

2.1.5 Other Miscellaneous Cables

Other miscelaneous cables needed include:

- CAT6 Network cables
- OM3 LC/LC Duplex Fiber Patch Cables (e.g. FS.com SKU 40233)
- IEC C13 to NEMA 5-15P Power Cords
- Assorted discrete wires for DC power, signal, 16-18AWG

Chapter 3

ATE Connections

3.1 Overview

On initial setup, there are a few fixed connections that need to be made to the ATE:

- Current Shunt: Connect this to a resistance standard for calibrations, or to a wire loop if not being used for current measurement
- $\pm 15V$ Supply: Connect this to a split supply, draws less than 400mA under full load
- Network: Connect to the local test network
- SD Card must be configured once, but does not require any regular changes
- Jumpers: Install jumpers in J8-9, J12-13, J19-20, JJ23-24, J30-31, J34-35, J41-42, and J45-J46 for normal operations.

For every device under test:

- Channel 1, 2, 3, and 4 Power Supply Cables: These cables connect to each of the micro-D Power Supply I/O connectors on the rear of the PSC
- Tuning Boards: Tuning boards must be installed to match the loads being tested.
- DCCT Cable must be connected for every PSC. There are two cables - one for 4-Channel units, and another for 2-Channel units.

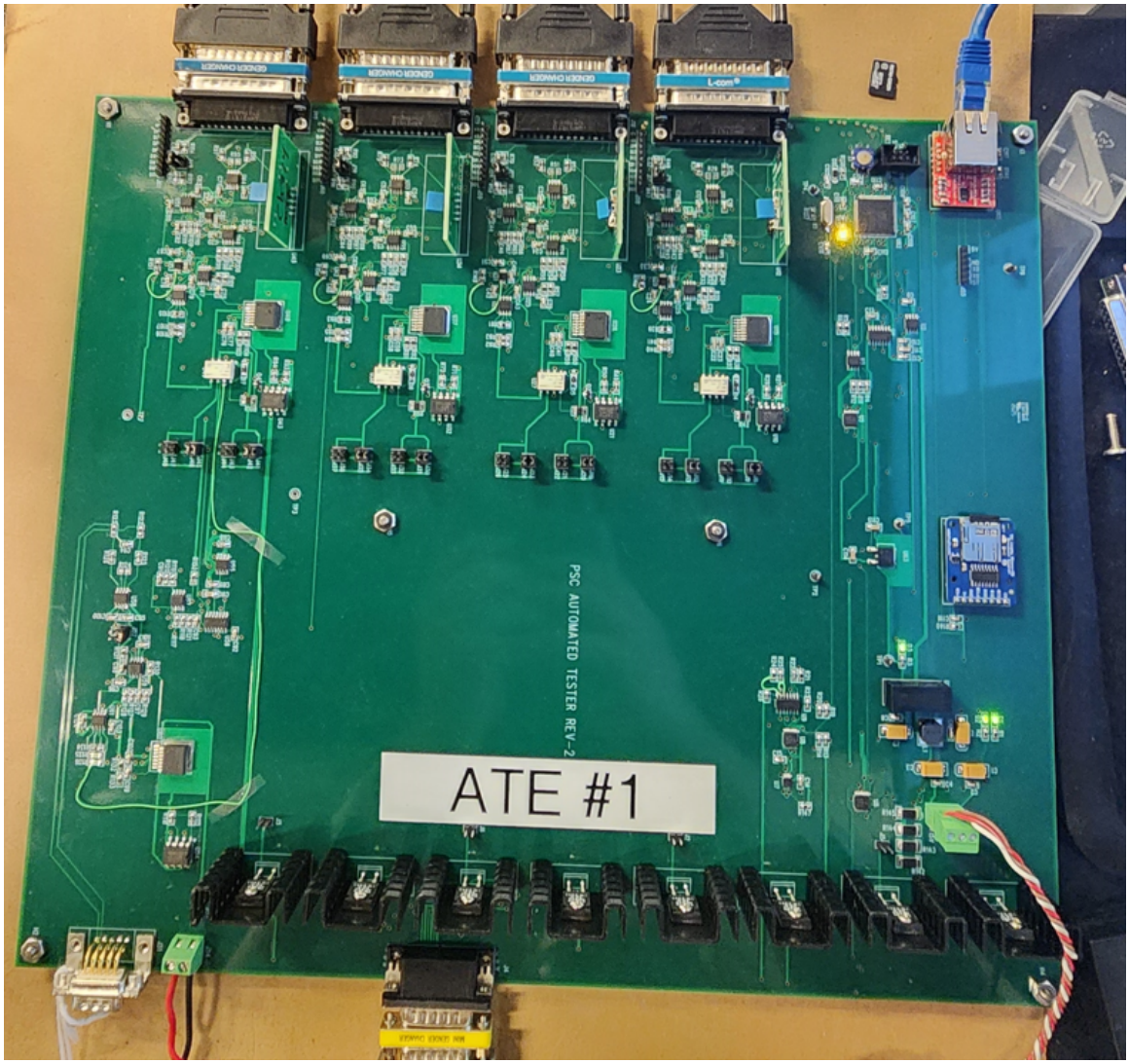
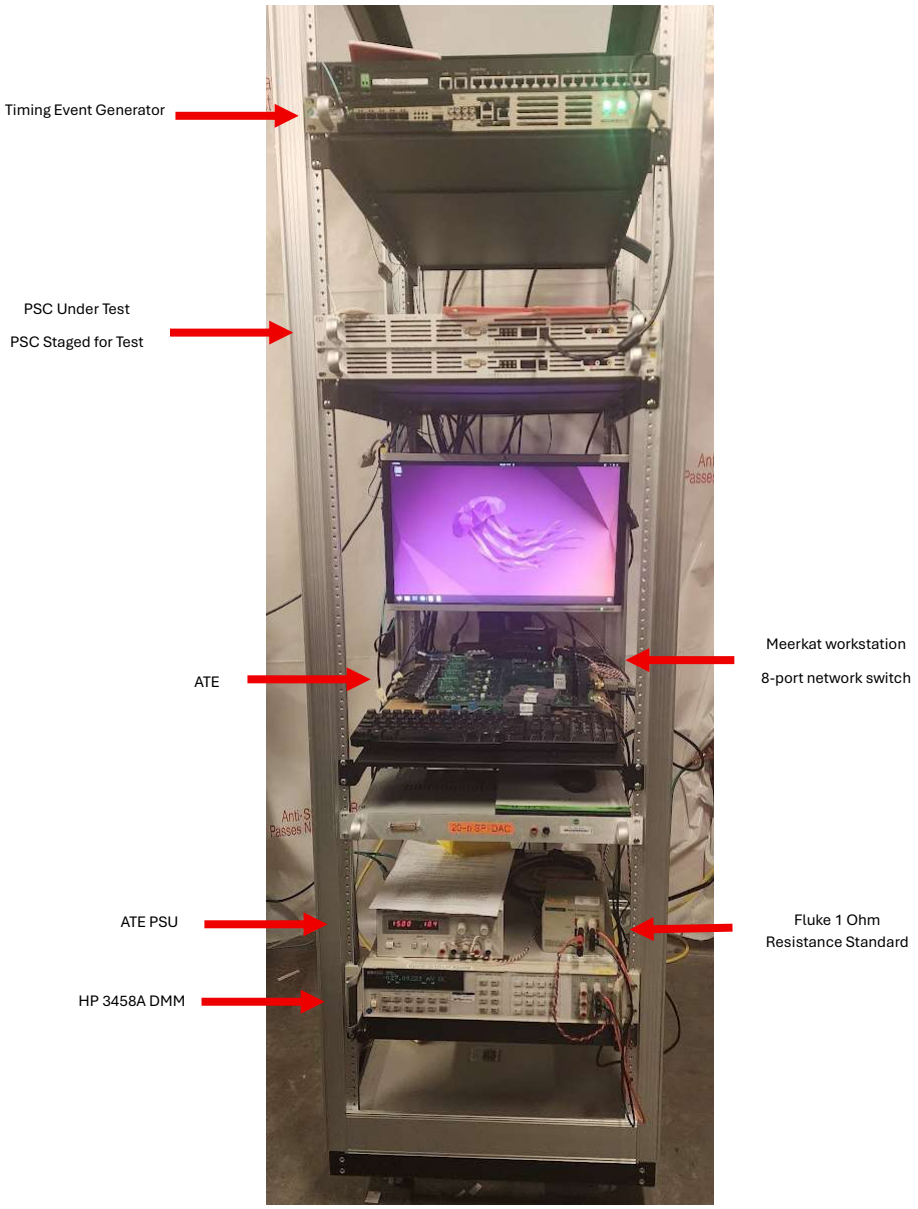


Figure 3.1: ATE #1

3.1.1 ATE Test Rack at NSLS-II



3.2 Connections

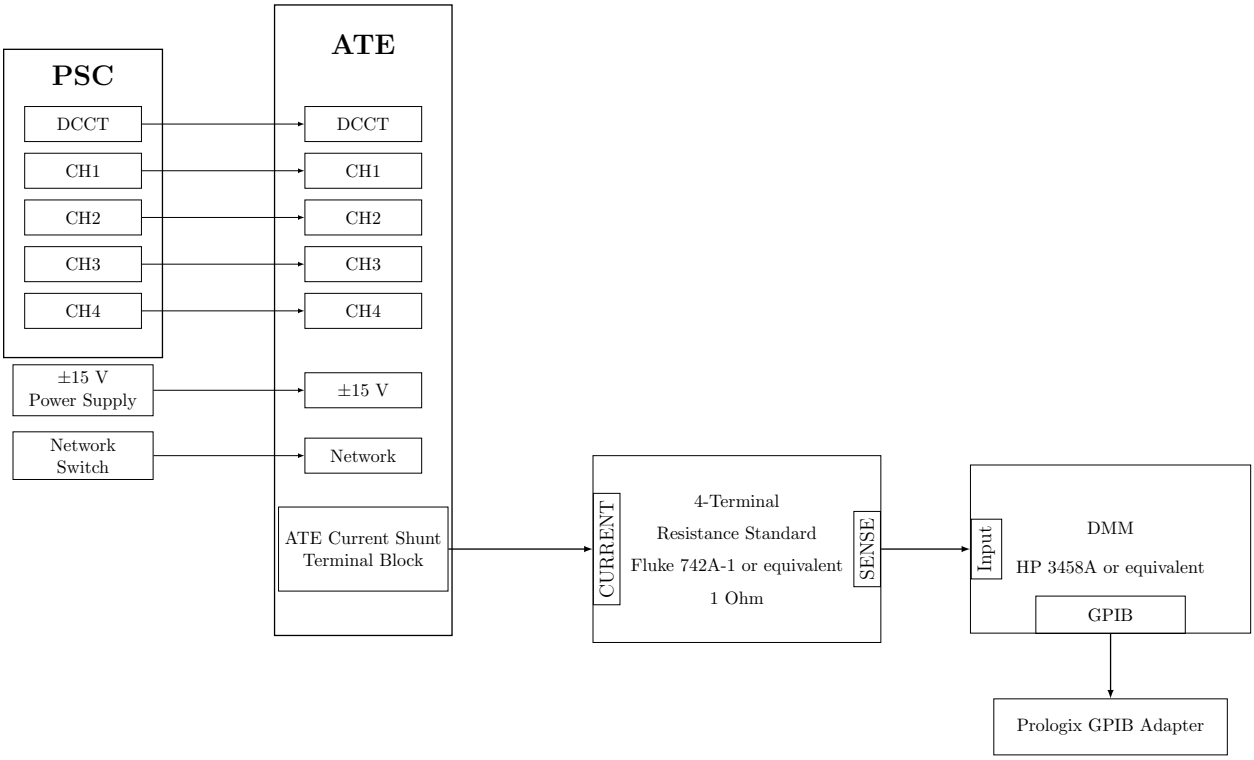


Figure 3.2: ATE Connection Block Diagram

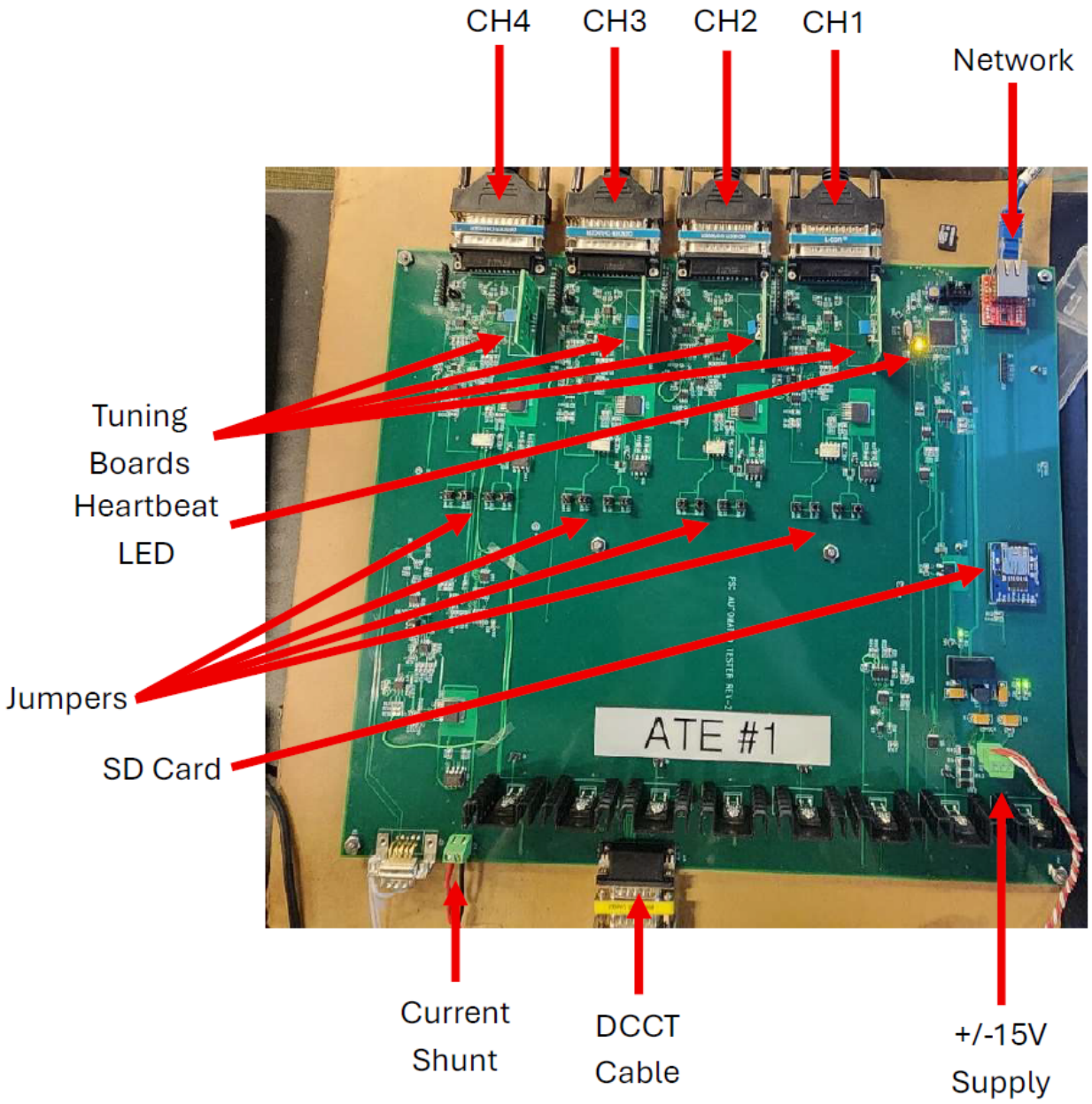


Figure 3.3: Annotated ATE Board

3.2.1 Current Measurement Loop/DMM Connection

The DMM is connected to the workstation via a GPIB adapter from Prologix. These are available in both USB and Ethernet forms.

If the DMM/Resistance standard is not used, a jumper must be connected across this terminal block on the ATE if CALDAC is to be used!

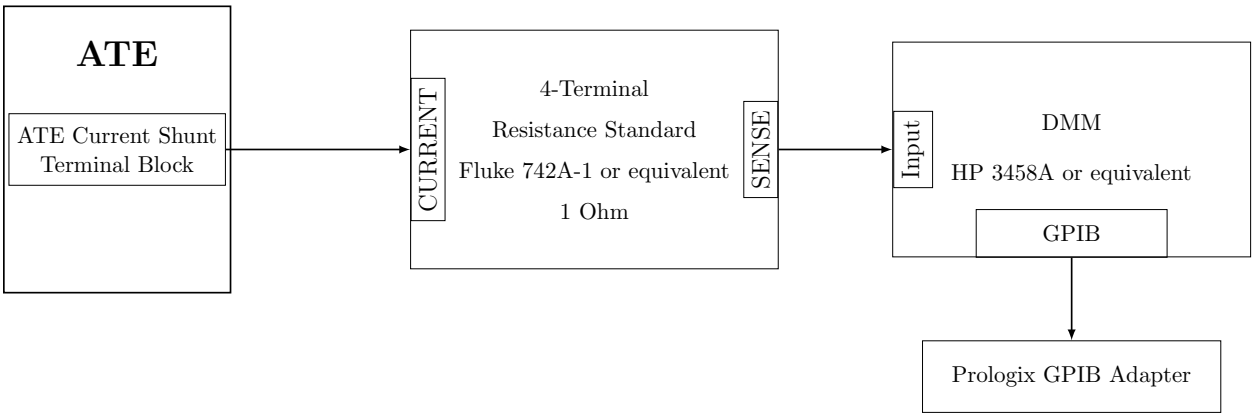


Figure 3.4: ATE → DMM Current Loop

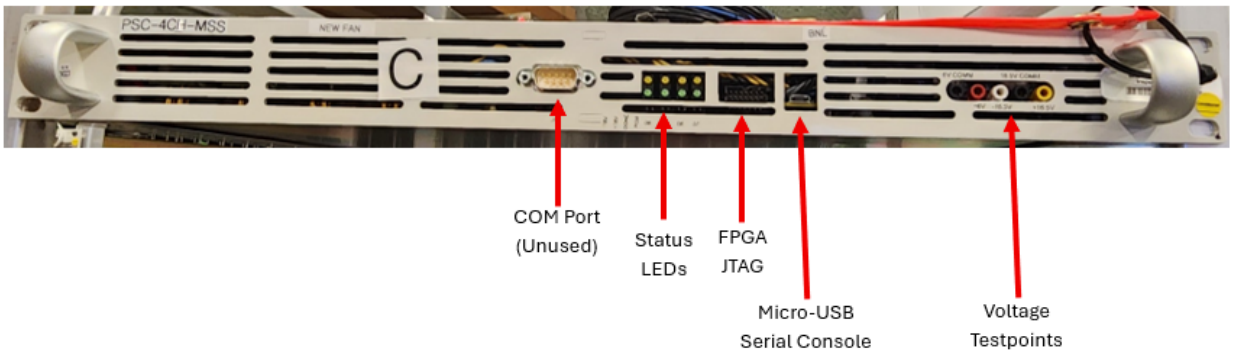
Chapter 4

zPSC Chassis Overview

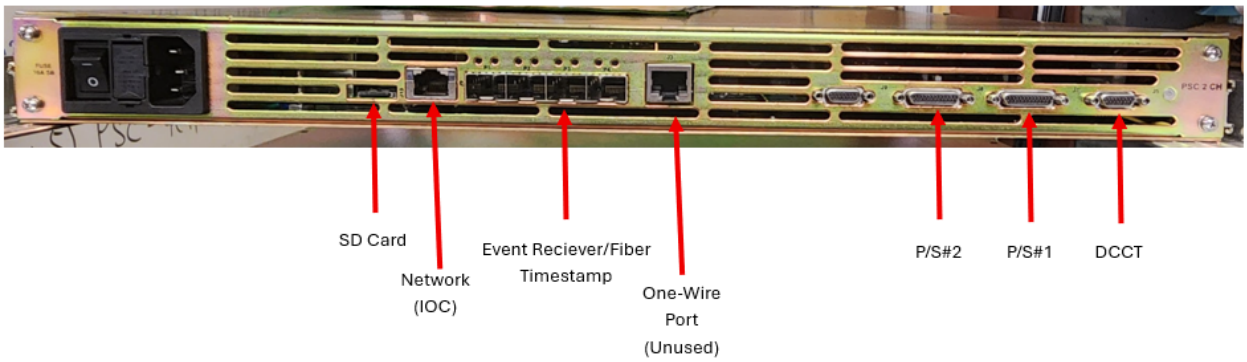
4.1 Introduction

4.2 Front Panels (2- and 4-Channel PSCs)

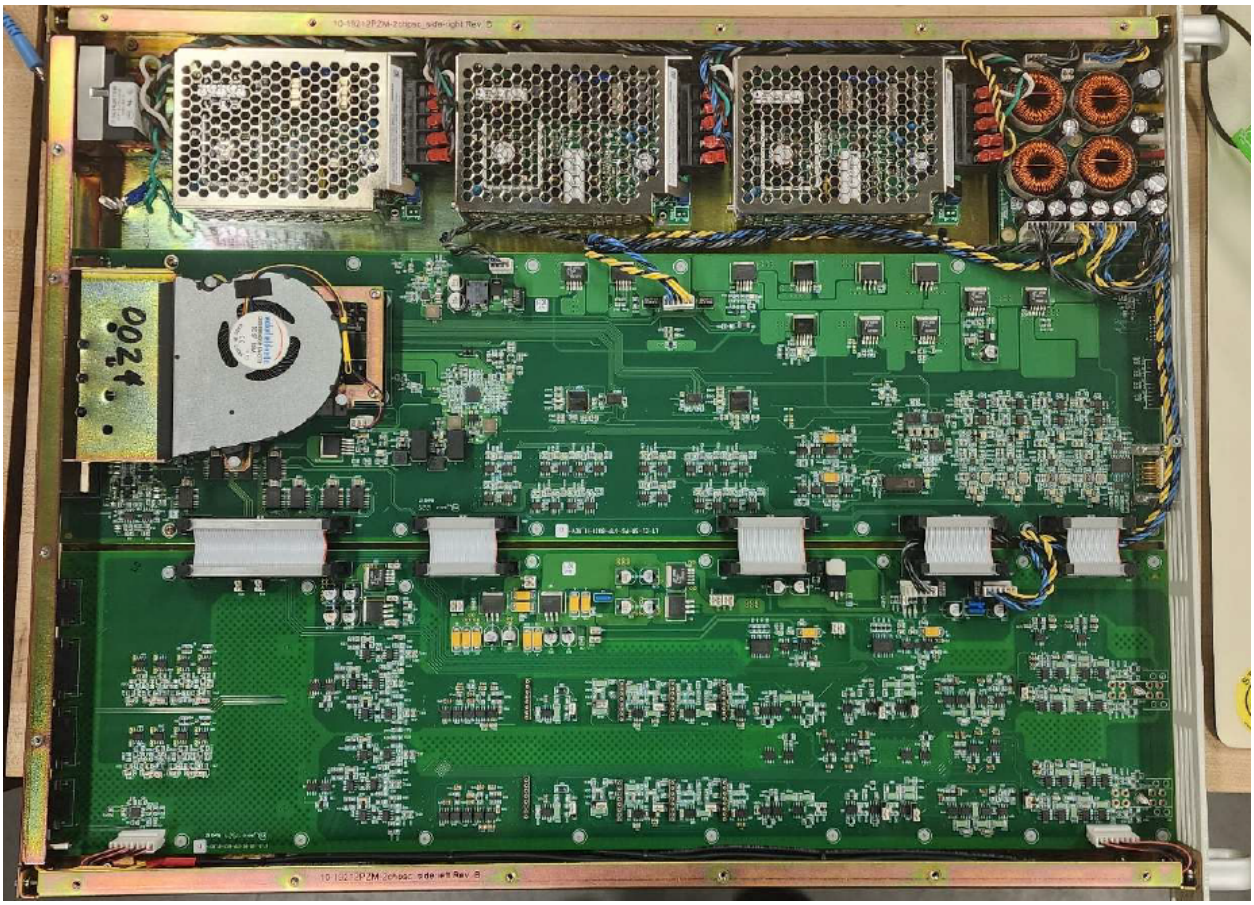
Both 2- and 4-Channel units share identical front panels:



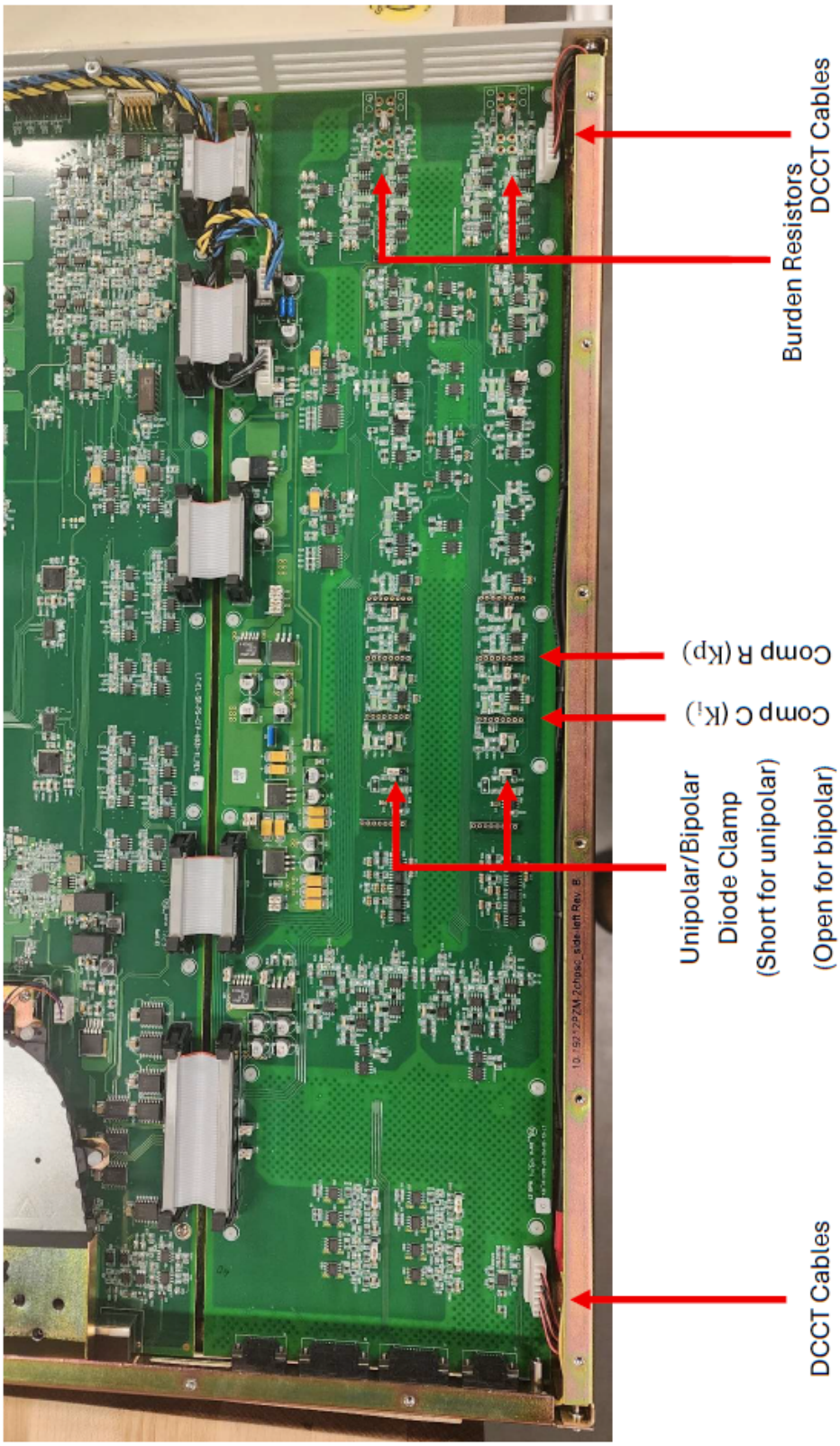
4.3 Rear Panel - 2 Channel



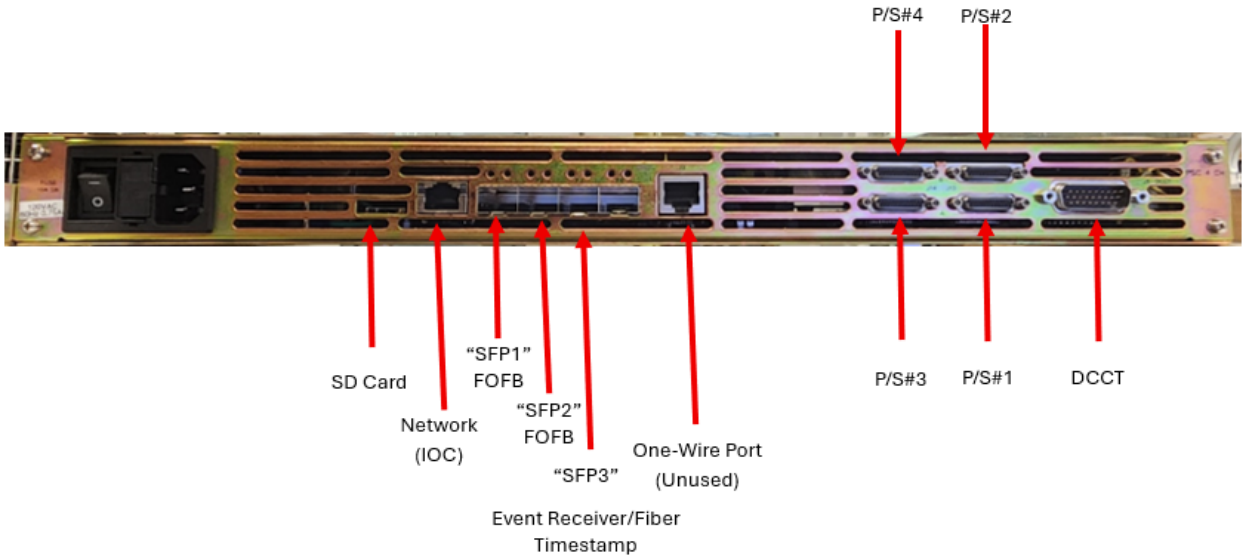
4.4 Internal Component Layout - 2 Channel



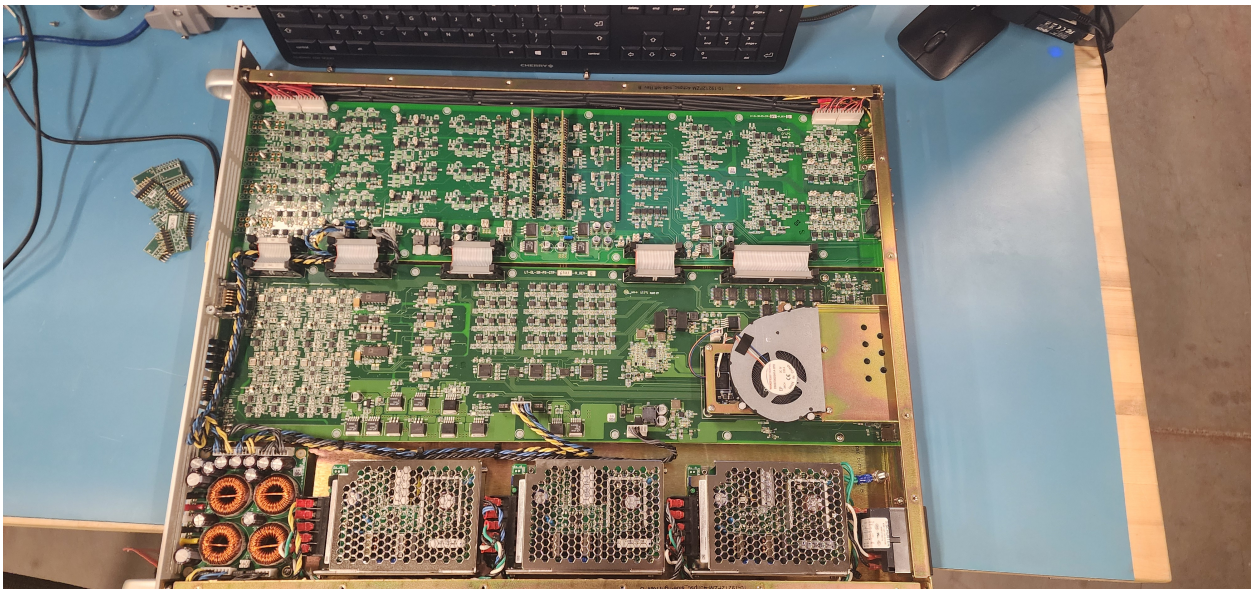
4.4.1 Analog Board Components



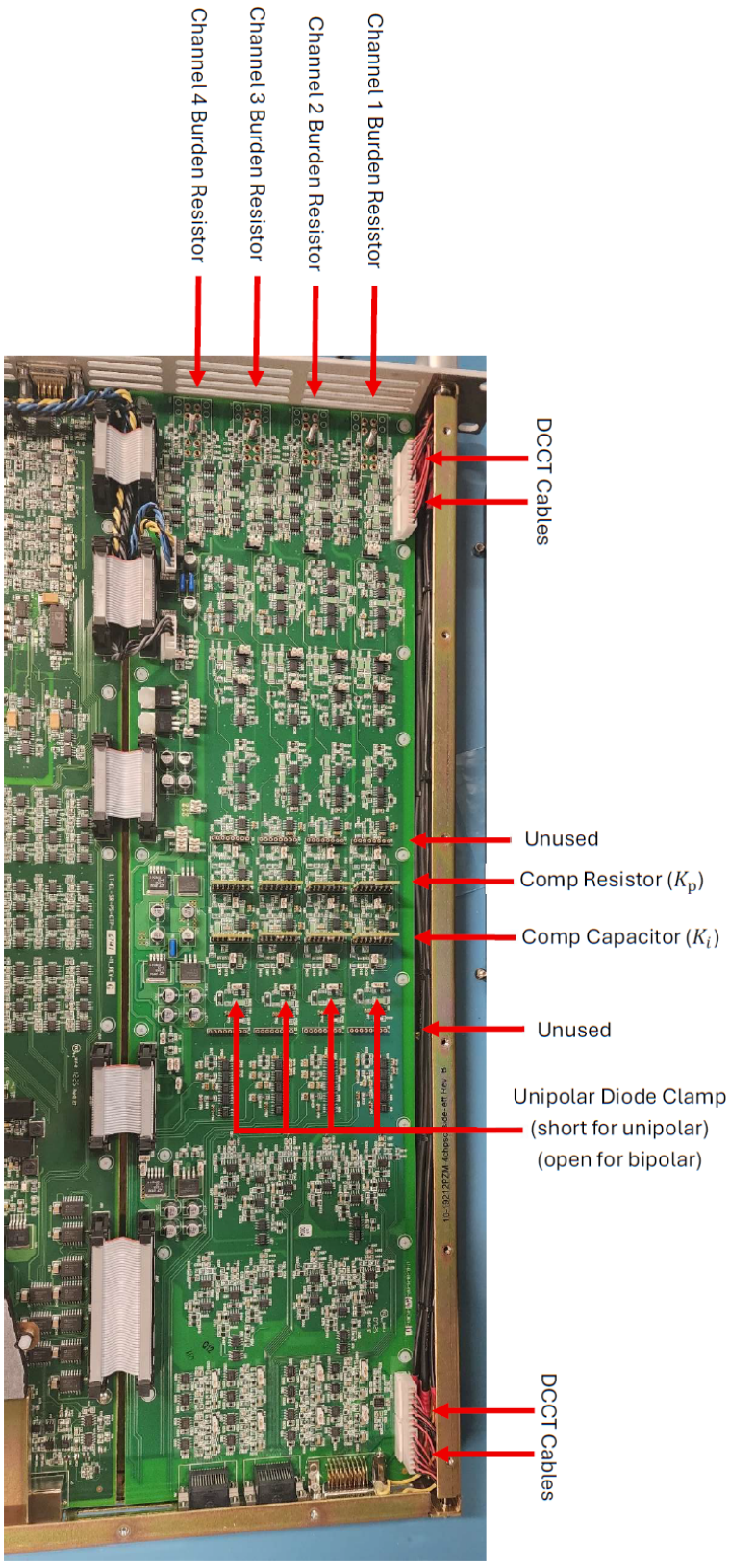
4.5 Rear Panel - 4 Channel



4.6 Internal Component Layout - 4 Channel



4.6.1 Analog Board Components



Chapter 5

EPICS Base and IOCs

5.1 Introduction

There are two IOCs associated with the calibration and test of the zPSC: The ATE IOC, and the zPSC IOC.

5.2 EPICS Installation Overview

EPICS installations performed by M. Capotosto loosely follow the below directory structure:

- `EPICS_BASE=/epics/base` or `/epics/base_[version]` or similar
- `SUPPORT=/epics/support` – or sometimes MODULES instead of SUPPORT is used.

  Modules required for the ATE and PSC are installed here:
  - seq: SNL-Sequencer
  - pscdrv: Portable Streaming Controller Driver
  - asyn: EPICS module for driver and device support
- `epics/iocs` - each IOC is cloned into this repository
- `epics/systemd-softioc` - This is an NSLS-II utility for managing IOCs as a system service

5.3 Installing EPICS Base and Required Modules

1. Install Dependencies for Build Install git, build-essential, and perl:
   `sudo apt -y install git build-essential perl libtirpc-dev re2c libevent-dev autoconf automake libtool pkg-config`

2. Create the EPICS user group, and the empty EPICS directory:
```
sudo groupadd epics
sudo usermod -aG epics $USER
newgrp epics
sudo mkdir -p /epics
sudo chown $USER:epics /epics
sudo chmod 2775 /epics
```

3. EPICS Base is cloned into `/epics/base` via:
```
git clone --recursive -b 7.0
   https://github.com/epics-base/epics-base.git /epics/base
cd /epics/base
```

4. Build EPICS:
```
make clean
make
```

5. Create the remaining directory structure:
```
mkdir -p /epics/iocs /epics/modules
```

6. Clone asyn:
   (a) `cd /epics/modules`
   (b) `git clone https://github.com/epics-modules/asyn.git asyn`
   (c) `cd /epics/modules/asyn`
   (d) Set up required files in `configure/RELEASE, configure/RELEASE.LOCAL`
   (e) Build with `make clean`, and `make`

7. Clone Sequencer:
   (a) `cd /epics/modules`
   (b) `git clone https://github.com/epics-modules/sequencer.git seq`
   (c) `cd /epics/modules/seq`
   (d) Set up required files in `configure/RELEASE, configure/RELEASE.LOCAL`
   (e) Build with `make clean`, and `make`

8. Clone PSC Driver:
   (a) `cd /epics/modules`
   (b) `git clone https://github.com/mdavidsaver/pscdrv.git pscdrv`

(c) `cd /epics/modules/pscdrv`

(d) Set up required files in `configure/RELEASE, configure/RELEASE.LOCAL`

(e) Build with `make clean`, and `make`

9. Clone Manage IOCs Utility:

(a) `cd /epics`

(b) `git clone https://github.com/NSLS2/systemd-softioc.git`

(c) Navigate to the repo directory `cd systemd-softioc`

(d) Make the install script executable and run:

```
sudo chmod +x install.sh
sudo ./install.sh
sudo usermod -aG epics softioc
newgrp
sudo chown softioc:epics /epics/iocs
sudo chmod 2775 /epics/iocs
```

5.4 Installing the ATE IOC

1. Change to IOC directory `cd /epics/iocs`
2. Clone the repo:
   `git clone https://github.com/capotostobnl/ALSu-PSC_ATE-ioc.git PSCtest`
3. `cd PSCtest`
4. `git switch Released,-UPC/BPC-Split-IOC-Ver-2`
5. Create the `RELEASE.local` file, `touch configure/RELEASE.local`
6. `echo -e "EPICS_BASE=/epics/base\nASYN=/epics/modules/asyn" > /epics/iocs/PSCtest/configure/RELEASE.local`
7. Build the IOC: `make clean`, `make`
8. Update the st.cmd file: `cd iocBoot/iocPSCtest`

(a) Edit the st.cmd file: `nano st.cmd`

(b) Update the EPICS_BASE variable if needed.

(c) Update the IPPORT to match the ATE. Repository has 10.69.26.4:5000 as default. Changing this later from the Phoebus/CSS panels may not work!

9. Find the next available port: `manage-iocs nextport`
10. Configure this in the config file: `nano config`

11. Set the `NAME=PSCtest`, `PORT=` your next port, `USER=softioc`, and `HOST=$HOSTNAME`
12. Update the directory path in the root-level st.cmd to point to the right binaries
13. Enroll in manage-iocs as a service: `sudo manage-iocs install PSCtest`
14. Enable auto-start on boot: `sudo manage-iocs enable PSCtest`
15. Check the status, it should be running! `manage-iocs status`
16. And if you want other information on the ioc: `manage-iocs report`

5.5 Installing the PSC IOC

1. Change to IOC directory `cd /epics/iocs`
2. Clone the repo:
   `git clone https://github.com/jamead/zpsc-ioc.git zpsc_ioc`
3. `cd zpsc_ioc`
4. Create the `RELEASE.local` file, `touch configure/RELEASE.local`
5. ```
   echo -e "EPICS_BASE = /epics/base\n
        PSCDRV = /epics/modules/pscdrv\n
        SEQ = /epics/modules/seq" >
         /epics/iocs/zpsc_ioc/configure/RELEASE.local
   ```
6. Build the IOC: `make clean`, `make`
7. Update the st.cmd file as needed: `cd iocBoot/ioczpsc`
8. Create the st.cmd file used by systemd-softioc:

   ```
   echo -e '#!/bin/bash\n\n
     cd "/epics/iocs/zpsc_ioc/iocBoot/ioczpsc"\n./st.cmd' > st.cmd
   ```

9. Make the new st.cmd executable: `chmod +x st.cmd`
10. Find the next available port: `manage-iocs nextport`
11. Configure this in the config file: `nano config`
12. Set the `NAME=zpsc_ioc`, `PORT=` your next port, `USER=softioc`, and `HOST=$HOSTNAME`
13. Update the directory path in the root-level st.cmd to the `iocBoot/ioczpsc/st.cmd`
14. Enroll in manage-iocs as a service: `sudo manage-iocs install zpsc_ioc`
15. Enable auto-start on boot: `sudo manage-iocs enable zpsc_ioc`
   ```

16. Check the status, it should be running! `manage-iocs status`
17. And if you want other information on the ioc: `manage-iocs report`

Chapter 6

CS-Studio - Phoebus

6.1 Introduction

CS-Studio Phoebus is installed using precompiled binaries. This application requires a Java runtime environment be installed.

The 'BOB' files - the user interface panels, are located in the IOC's directories, `/epics/iocs/PSCtest/gui` and `/epics/iocs/zpsc_ioc/gui`

Behavior of some features of Phoebus must be modified by the shell script used to launch Phoebus, an example is included with the below installation instructions to modify the behavior of the 'Home' button, used to bring up the custom 'PSC Launcher' panel using the Home button, instead of having to manually navigate to the file when needed.

6.2 Installing Phoebus

1. If not installed, install Java, e.g.:
   ```
   sudo apt update && sudo apt install openjdk-17-jdk
   ```
2. Create the directory: `sudo mkdir /opt/phoebus`
   ```
   sudo chown $USER:epics /opt/phoebus
   chmod 2775 /opt/phoebus
   ```
3. Download Phoebus and extract to the `/opt/phoebus` directory
   https://github.com/ControlSystemStudio/phoebus/releases/download/v4.7.4/phoebus-4.7.4-linux.tar.gz

   ```
   cd /opt/phoebus
   wget https://github.com/ControlSystemStudio/phoebus/releases/download/
   v4.7.4/phoebus-4.7.4-linux.tar.gz
   ```

```
tar -xvzf phoebus-4.7.4-linux.tar.gz -C . --strip-components=1
```

4. Make the shell script executable: `chmod +x phoebus.sh`
5. Copy in necessary configuration changes from the template:
   ```
   git clone
   https://github.com/capotostobnl/Phoebus-ALSU.git /opt/phoebus/config
   ```
6. overwrite the defaults with the new template: `mv config/* .`
7. Update paths as necessary in the `settings.ini`
8. Alias Phoebus to run via command `run-phoebus`:
   ```
   echo 'alias run-phoebus="/opt/phoebus/phoebus.sh"' >> ~/.bashrc
   source ~/.bashrc
   ```
9. Now test! Try run-phoebus from a terminal, and then click the 'Home' button!

Chapter 7

Networking

7.1 Overview

The default configuration for these devices is to use static IP addresses on the 10.69.26.0/23 subnet. Deviating from this will require updates be made to the IOCs for the PSC and ATE, as well as changes to some parts of the calibration scripts, and, if applicable, the FOFB Test module of the functional test script set.

The network topology is generally as follows:

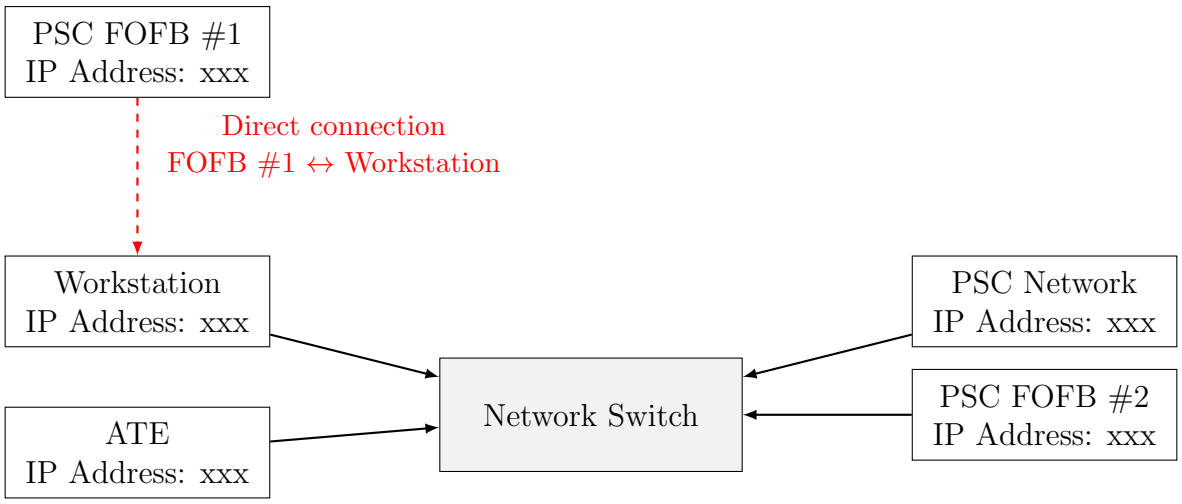


Figure 7.1: Network Topology with Clean FOFB Direct Link

Chapter 8

Installing and Configuring Minicom

8.1 Overview

Minicom is the serial terminal emulator of choice for NSLS-II, but many other options exist such as Teraterm for Windows, or PuTTY for cross platform.

8.2 Installation in a Linux environment

User Permissions

Access to the serial port by default requires privilege escalation. This issue can be sidestepped by adding the user accounts which need to access the serial ports to the dialout (or uucp for RHEL) user group.

This is also required later for use of the calibration script, if a USB-based GPIB adapter is used, otherwise those scripts must run as root, which can cause permissions issues with reports.

To add a user to the dialout group: `sudo usermod -aG dialout $USER`
Then to test: `newgrp dialout`, then `groups`. You should see dialout in your groups.

You may need to log out and log back in to ensure this persists in new terminal windows.

8.2.1 Installing Minicom

Installation of Minicom is simple, in a Debian-based distribution, use apt:
`sudo apt-get update && sudo apt-get install -y minicom`

8.3 Configuring Minicom

Create a default configuration if one does not exist for Minicom:

Open the a text file as root: `sudo nano /etc/minicom/minirc.dfl` and enter the following (One line per command, whitespace/tabs inside the line isn't critical)

```
pu baudrate         115200
pu bits             8
pu parity           N
pu stopbits         1
pu rtscts           No
pu xonxoff          No
```

This disables hardware and software flow control, and sets baud rate, parity, and stop bits.

8.4 Running Minicom

You are now ready to run Minicom. Find the PSC's serial port. With no PSC connected, you run: `ls /dev/`

Then plug in the PSC, and re-run the command; it should appear as a USB device, e.g. ttyUSB2. Search the list for which one new device will suddenly appear in the list.

Then: `minicom -D /dev/ttyUSB2`, replacing ttyUSB2 with whatever your PSC's port is.
A serial terminal should open; press return, and the PSC's menu should now print on the screen.

Chapter 9

Python Installation and Configuration

9.1 Overview

Python3 is required for the calibration and test scripts to be run for the PSCs. Below sections will describe the process for installing Python3, and handling package management and virtual environment management, if desired.

9.2 Installing and Configuring Python, and the Python Environment

9.2.1 Installing Python

Many downstream Debian-based distributions come bundled with Python3. If yours does not, install Python3 via: `sudo apt update && sudo apt install python3`

9.2.2 Python Package Management

Installing Pip

These scripts were written using Python3. It is assumed that Python3 is already installed on your workstation.

A Pip requirements file is included in the repository for ease of package management, but pip may not come bundled with the OS; to install Pip:
`sudo apt install python3-pip`

Installing Packages

Packages can be simply installed via pip now.

If within a virtual environment, use: `pip install -r requirements.txt`
If not using environments: `python3 -m pip install -r requirements.txt`
If you run into permissions issues: `python3 -m pip install --user -r requirements.txt`

Python Virtual Environments

It may be desireable to use a virtual environment, e.g., venv or Conda, especially if other scripts are used on this machine that may have overlapping or conflicting packages installed. My personal preference is for venv, however Conda is used by NSLS-II at the organizational level. venv is a simpler and light weight tool, if it is needed only for running Python projects.

venv can be installed with:
`sudo apt install python3-venv`

The environment can then be activated with `python3 -m venv my_venv_name`. You can then activate the environment with `source my_venv/bin/activate`.

Chapter 10

Calibration and Test Script Installation

10.1 Overview

The repository for the calibration and test scripts can be found at: `https://github.com/capotostobnl/NSLS2_zPSC_Calibration_and_Test`

Details regarding the use of the script will come in the following chapters on Calibration and on Functional Tests, though all scripts are launched from this one repository's 'launcher.py' script.

10.2 Cloning the Script Repository

1. Choose a location for the repo, and go there: `cd /`
2. Clone the repository: `git clone https://github.com/capotostobnl/NSLS2_zPSC_Calibration_and_Test`

10.3 Creating the Virtual Environment

If you choose to use a virtual environment, create it and then install the dependencies. If you choose to not use venv, you can skip to installing dependencies in the next section.

1. Navigate to the repository and then: `python3 -m venv .venv`
2. Activate the environment: `source .venv/bin/activate`

10.4 Installing Dependencies

Required Python modules can be installed via the requirements.txt from the repository via: `python3 -m pip install -r requirements.txt`

Chapter 11

Initial Booting of the PSC and Configuration of the EEPROM

11.1 Overview

This section will discuss the steps required for the first time booting a new PSC, and initializing the EEPROM.

The EEPROM must be configured before many functions will work.

It is important to note that once the EEPROM is configured, many parameters of the PSC will still provide inconsistent or invalid data until the QSPI values are initialized properly via the scripts outlined in chapter 12.

Chapter 12

Running the Calibration and Test Script

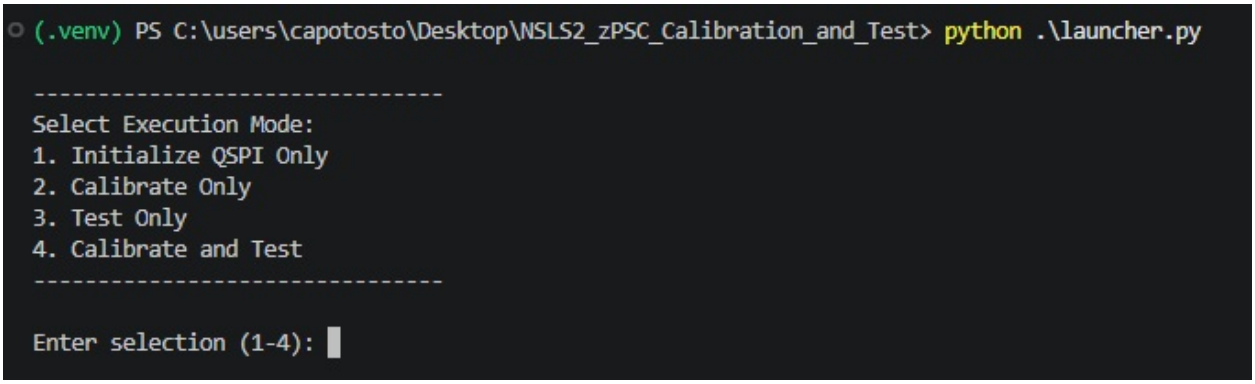
12.1 Overview

This section will describe how to run and use the Calibration and Test Script, assuming it is configured properly. The following chapters will go into detail on adding new PSC models, and specific details regarding the calibration side and test side of the scripts.

> **The EEPROM must be configured in the PSC prior to using any of these scripts. Failure to configure the EEPROM, or incorrectly configuring it, will cause issues with the script at even the first menu selection stages.**

12.2 Running the Script: Launcher Menu

To run the script, simply execute the launcher.py script from the repository root directory: `python3 launcher.py`
A menu will appear in your terminal.



Choose an option from the menu.

1. Initialize QSPI Only
   This option will set the QSPI parameters normally written at the end of calibration, so values such as OVP/OCP, scale factors, and such, will be written to the flash. All gains are written as 1.00, and all offsets written as 0.0. This will overwrite any values stored in QSPI!

   This is especially useful for initial testing, where you may want to verify all channels work visually, before committing the time to calibrate and test.

2. Calibrate Only
   This option will perform a calibration only, without any tests. This will write all QSPI values at the end of the calibration for each channel.

3. Test Only
   This option will perform only the functional tests, without a calibration. This makes no changes to the QSPI.

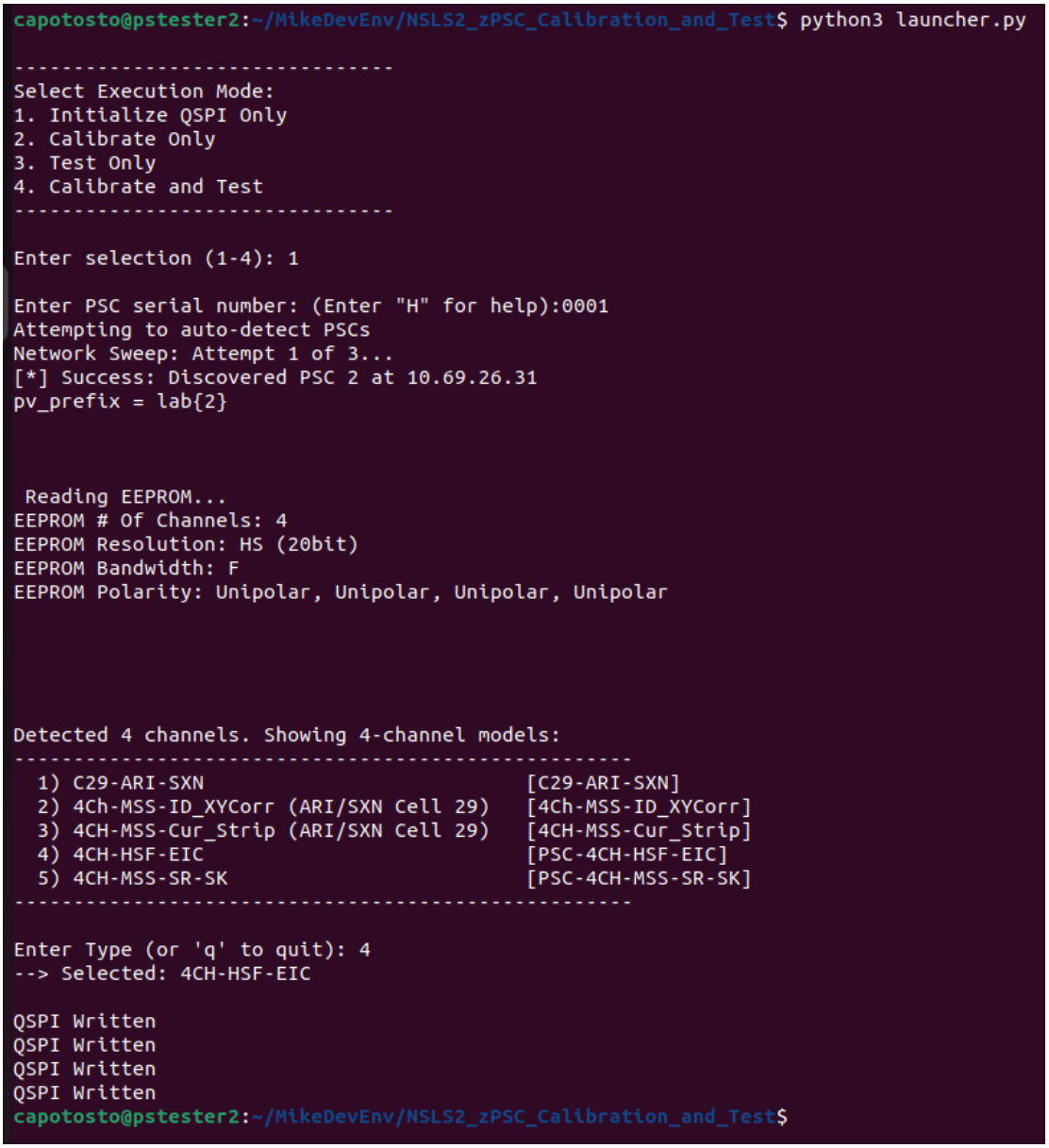
4. Calibrate and Test
   This option will run a full calibration, and immediately upon completion of the calibration, it will begin the functional test. This allows a calibration and test cycle to be started, and then left alone; so long as the calibration doesn't fail in a way that exits the script, the test can be left unattended for the duration and it will complete without any further user input once it has began (with the exception of the Fast Orbit Feedback Test, which requires a user password)

12.2.1 Option 1: Initialize QSPI Only

An example screenshot is provided below.
After selecting Menu Option 1, Initialize QSPI Only, the script will sweep the network to detect PSCs. If only one PSC pings, it'll automatically select this unit. If it times out, it'll prompt for the PV prefix of the PSC.

The script reads the EEPROM configuration, and trims the option list to those PSCs that match the EEPROM.

In this example, a 4Ch High-Stability Fast EIC PSC was being configured, so we chose Option 4. Each channel's QSPI was then written, and the script exited gracefully.



12.2.2 Option 2: Calibrate Only

An example screenshot is provided below.
After selecting Menu Option 2, Calibrate Only, the script will sweep the network to detect PSCs. If only one PSC pings, it'll automatically select this unit. If it times out, it'll prompt for the PV prefix of the PSC.

The script reads the EEPROM configuration, and trims the option list to those PSCs that match the EEPROM.

In this example, a 4Ch High-Stability Fast EIC PSC was being calibrated, so we chose Option 4.